Assistente del laboratorio

Chatbot da terminale con grafo LangGraph, modello locale in LM Studio, strumento orologio e avvio con Docker Compose

Guida riassuntiva e teorica per preparare l'esposizione

Dato	Valore
Progetto	`Lezione 4/assistente-laboratorio/`, branch `main`
Commit raggiunto	`bbe18d9` · docs: completa README e CLAUDE.md dopo la verifica finale
Modello	`google/gemma-4-e2b` (GGUF, Q4_K_M) servito da LM Studio sull'host
Tecnologie	Python 3.13 · LangGraph 1.2.18 · langchain-openai 1.7.3 · Docker Compose v5
Test	69 automatici senza modello · 3 prove reali LM Studio · 1 prova reale Docker
Data	2 novembre 2026

Indice

1. Il progetto in breve
2. Teoria · LLM locali, LM Studio e API compatibile con OpenAI
3. Teoria · LangGraph: stato, nodi, archi e cicli
4. Teoria · Tool calling: il modello chiede, il programma esegue
5. Teoria · Tempo, fusi orari IANA e ISO 8601
6. Teoria · Docker e Docker Compose
7. Architettura del progetto
8. Il grafo nel dettaglio
9. Un turno passo per passo
10. Lo strumento orologio
11. Cronologia, rollback ed errori
12. Configurazione
13. Test e verifiche
14. Metodo di lavoro: CLAUDE.md, Superpowers e Git
15. Risultati, misure e decisioni
16. Limiti, problemi aperti e prossimi passi
17. Come impostare l'esposizione (scaletta e demo)
18. Domande probabili e risposte
19. Glossario

1. Il progetto in breve

L'**assistente del laboratorio** è un'applicazione Python da terminale che conversa con un modello linguistico eseguito in locale da **LM Studio**. Il flusso dell'applicazione non è scritto in modo lineare: è descritto da un **grafo LangGraph** esplicito, con uno stato condiviso, dei nodi e una funzione di instradamento. Quando l'utente chiede l'ora, il modello non la inventa: **richiede lo strumento orologio**, il programma lo esegue davvero e il risultato torna al modello, che formula la risposta finale.

- **Grafo esplicito**: START → llm → (strumenti → llm)* → END, con limite di passaggi per turno.
- **Cronologia di sessione**: il modello riceve a ogni turno tutta la conversazione; alla chiusura viene persa.
- **Strumento orologio** `ora_corrente(fuso_orario?)` con fuso IANA esplicito, visibile in chat e nei log.
- **Docker Compose**: l'applicazione gira nel container, LM Studio resta sull'host.
- **Test** automatici indipendenti dal modello e **prove reali** separate, attivate esplicitamente.

Esempio reale di sessione (28 ottobre 2026; le righe [strumento] sono mandate a capo qui per leggibilità, nel terminale sono su una riga sola):

```
Assistente del laboratorio — modello google/gemma-4-e2b su http://localhost:1234/v1
Scrivi «esci» per terminare.
Tu> Che ore sono adesso a Tokyo?
[strumento] ora_corrente(fuso_orario="Asia/Tokyo")
            → 2026-10-28T20:08:51+09:00 (Asia/Tokyo), mercoledì
Assistente> Adesso sono le 20:08:51 di mercoledì 28 ottobre 2026 a Tokyo.
Tu> Che giorno della settimana è oggi a New York?
[strumento] ora_corrente(fuso_orario="America/New_York")
            → 2026-10-28T07:09:08-04:00 (America/New_York), mercoledì
Assistente> Oggi a New York è mercoledì.
Tu> esci
```

Requisiti obbligatori della consegna (slide 26) e stato

Requisito	Evidenza	Stato
Grafo con LLM e cronologia	Conversazione su più turni: al secondo turno il modello ricorda «Ada»	Fatto
Strumento orologio	Richiesta, esecuzione e risultato nella riga [strumento] e nei log INFO	Fatto
Docker Compose	Container collegato a LM Studio; prova -m docker superata	Fatto (chat con TTY da confermare a mano)
Test	69 automatici + prove reali con marcatori lmstudio e docker	Fatto
Documentazione	README, .env.example e CLAUDE.md coerenti	Fatto
Repository condivisa	Commit Conventional leggibili; push eseguito dallo studente	Push da fare

2. Teoria · LLM locali, LM Studio e API compatibile con OpenAI

Che cosa fa un LLM

Un **Large Language Model** è una rete neurale (un Transformer) addestrata a prevedere il **token successivo** di un testo. Un token è un frammento di parola. Generare una risposta significa ripetere la previsione token dopo token finché il modello produce un segnale di fine. Da questo derivano tre proprietà che guidano tutto il progetto:

- **Il modello è senza stato**: ogni chiamata è indipendente. Se vogliamo che «ricordi» la conversazione, dobbiamo rinviargli ogni volta la cronologia. La «memoria» è dell'applicazione, non del modello.
- **Il modello non ha accesso al mondo**: non legge l'orologio, non apre file, non naviga. Può solo produrre testo plausibile. Per un dato corrente come l'ora serve uno **strumento** eseguito dal programma.
- **Il modello è probabilistico**: la stessa domanda può dare risposte diverse. Per questo le prove reali vanno ripetute e i risultati riportati con la loro frequenza (es. 12 su 12).

LM Studio

LM Studio è un'applicazione desktop che scarica ed esegue modelli in formato **GGUF** sul computer locale e li espone tramite un server HTTP con un'**API compatibile con OpenAI**, di default su `http://localhost:1234/v1`. Gli endpoint usati dal progetto sono due:

Endpoint	Uso nel progetto
`GET /v1/models`	All'avvio: verificare che `LMSTUDIO_MODEL` esista davvero. L'identificativo del modello non si inventa: è quello restituito qui.
`POST /v1/chat/completions`	A ogni passaggio del nodo llm: invio di messaggi e descrizione degli strumenti, ricezione della risposta o delle richieste di tool.

Poiché l'API è compatibile con OpenAI, il progetto usa il client standard **ChatOpenAI** di langchain-openai cambiando solo `base_url`. LM Studio non richiede una chiave: il codice usa il segnaposto fisso `"lm-studio"`, che non è una credenziale.

Messaggi e ruoli

Ruolo	Classe LangChain	Contenuto
system	SystemMessage	Istruzioni stabili: rispondi in italiano, per data e ora usa sempre ora_corrente, passa il fuso IANA del luogo.
user	HumanMessage	La domanda dell'utente.
assistant	AIMessage	La risposta del modello, oppure una o più richieste di strumento (tool_calls) con id.
tool	ToolMessage	Il risultato dello strumento, collegato alla richiesta tramite tool_call_id.

Il modello usato

`google/gemma-4-e2b` è una variante piccola di Gemma 4 (circa 2 miliardi di parametri effettivi), quantizzata **Q4_K_M**: i pesi sono memorizzati con circa 4 bit invece di 16, così il modello occupa meno memoria e gira su un PC, con una piccola perdita di qualità. In LM Studio è caricato con un contesto di 8192 token e dichiara la capacità *tool_use*.

> **Da sapere: il modello «pensa» prima di rispondere**
> Le misure hanno mostrato che Gemma 4 lavora in modalità *thinking*: prima della risposta genera un ragionamento nascosto (in inglese) di circa 140–195 token, cioè circa il 90% dei token generati. A circa 16 token al secondo è la causa principale dell'attesa (vedi sezione 15).

3. Teoria · LangGraph: stato, nodi, archi e cicli

In un client semplice il percorso è scritto direttamente nel codice: leggi la domanda, chiama il modello, stampa. Con **LangGraph** le operazioni e i passaggi vengono descritti come un **grafo**, che può avere percorsi alternativi e cicli. La chiamata al modello diventa *un'operazione* del programma, non l'intera applicazione. LM Studio continua a eseguire il modello; LangGraph gestisce il flusso dell'applicazione Python.

Elemento	Significato	Nel progetto
Stato	Dati condivisi durante l'esecuzione (un TypedDict)	`messages`, `passaggi`, `limite_raggiunto`
Nodo	Funzione che legge lo stato e restituisce un aggiornamento parziale	`llm` e `strumenti`
Arco	Collegamento fisso tra due nodi	START → llm, strumenti → llm
Arco condizionale	Una funzione sceglie il nodo successivo in base allo stato	`instrada()`: «strumenti» oppure «fine»
START / END	Ingresso e uscita del grafo	L'uscita restituisce lo stato finale al chiamante
Reducer	Regola con cui un aggiornamento si combina con lo stato	`add_messages`: aggiunge i messaggi invece di sostituire la lista

Il ciclo di vita di un grafo

- **Definire lo stato**: quali dati attraversano il grafo.
- **Registrare i nodi**: `add_node("llm", funzione)`.
- **Collegare i nodi**: `add_edge` e `add_conditional_edges`.
- **Compilare**: `compile()` produce un grafo eseguibile. La compilazione non chiama il modello e non dimostra che il comportamento sia corretto.
- **Invocare**: `invoke(stato_iniziale)` restituisce lo stato finale; `stream(..., stream_mode="updates")` mostra quali nodi sono stati eseguiti (usato nei test).

Il reducer add_messages

Un nodo restituisce solo i campi che cambia. Per la maggior parte dei campi il nuovo valore sostituisce il vecchio (per esempio `passaggi`). Per `messages` invece il reducer `add_messages` **aggiunge** i nuovi messaggi in coda (o aggiorna quelli con lo stesso id). Senza questo reducer, il nodo llm cancellerebbe per errore tutta la cronologia scrivendo solo la sua risposta.

Dal chatbot all'agente: il ciclo con strumenti

Un grafo con un arco che torna indietro (strumenti → llm) è un **ciclo**: il modello può chiedere uno strumento, ricevere il risultato e decidere se chiederne un altro o rispondere. È lo schema di base degli **agenti**. Ogni ciclo deve avere una condizione di uscita: qui la fine arriva quando il modello non chiede più strumenti, oppure quando si raggiunge `MAX_PASSAGGI` chiamate al modello nello stesso turno.

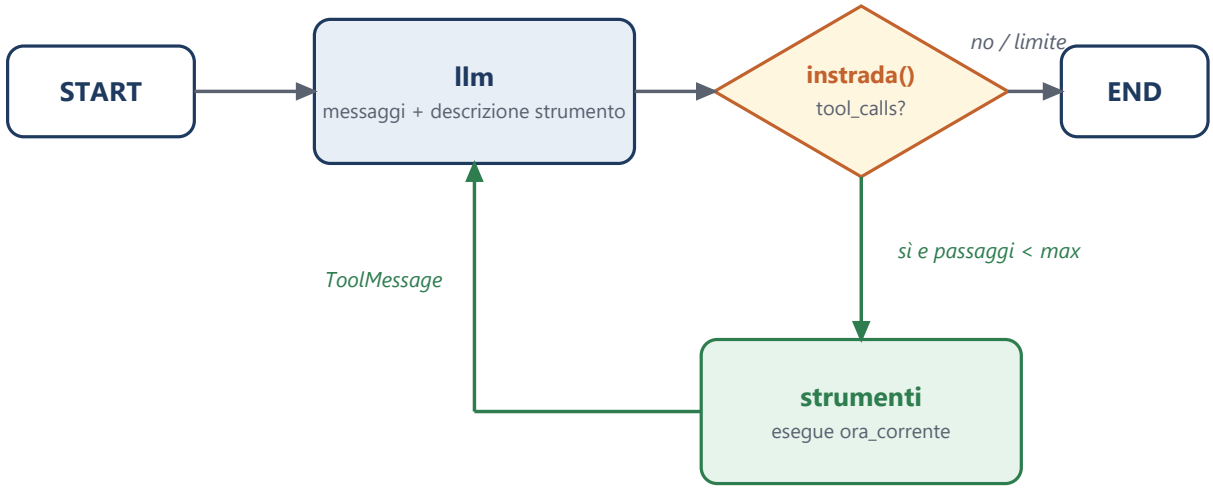


Figura 1 · Il grafo dell'assistente. La funzione di instradamento è scritta nel progetto (non è il condizionale precostruito di LangGraph), così la decisione è leggibile e registrata nei log.

4. Teoria · Tool calling: il modello chiede, il programma esegue

Il **tool calling** (o function calling) permette a un modello di usare funzioni del programma. Il punto chiave per l'esposizione: **il modello non esegue nulla**. Produce soltanto una richiesta strutturata; l'esecuzione avviene nel codice Python.

Passo	Chi	Che cosa succede
1	Programma	Invia al modello i messaggi e, nel campo `tools`, la descrizione dello strumento in JSON Schema: nome, descrizione, parametri.
2	Modello	Se serve, risponde con `tool_calls`: nome dello strumento, argomenti in JSON e un id univoco (es. call_1).
3	Programma	Il nodo strumenti esegue la funzione reale e aggiunge un ToolMessage con lo stesso `tool_call_id`.
4	Modello	Riceve il risultato associato alla sua richiesta e formula la risposta finale (o chiede un altro strumento).

Lo schema che il modello riceve davvero per l'orologio (generato da LangChain a partire dalla funzione Python):

```
{"type": "function",
 "function": {
   "name": "ora_corrente",
   "description": "Restituisce la data, l'ora e il giorno della settimana attuali.
       Usala ogni volta che servono data, ora o giorno correnti, senza inventarli.
       Il parametro fuso_orario è un nome di fuso IANA, per esempio Europe/Rome,
       Asia/Tokyo, America/New_York, UTC. ...",
   "parameters": {
     "type": "object",
     "properties": {
       "fuso_orario": {
         "anyOf": [{"type": "string"}, {"type": "null"}],
         "default": null,
         "description": "nome di fuso IANA del luogo di cui si chiede l'ora, per esempio
             Asia/Tokyo per Tokyo ... Indicalo sempre quando la domanda nomina una città
             o un paese; omettilo solo per l'ora locale."}}}}}
```

Perché la descrizione conta

La descrizione è l'unica cosa che il modello sa dello strumento: è il suo **contratto**. Nel progetto l'abbiamo misurato: senza la descrizione del parametro, alla domanda sull'ora a Tokyo il modello ometteva `fuso_orario` in 5 casi su 17 (e otteneva l'ora di Roma); con la descrizione del parametro e una frase nel prompt di sistema la prova è passata 12 volte su 12.

Casi difficili gestiti

- **Strumento inesistente** (es. «meteo»): ToolMessage che inizia con `Errore:`, il turno prosegue.
- **Argomenti sconosciuti** (es. `{"timezone": "Asia/Tokyo"}`): lo schema pydantic li ignorerebbe in silenzio e lo strumento darebbe l'ora di Roma. Il nodo li confronta con quelli ammessi e restituisce un errore.
- **Argomenti che non sono JSON valido**: LangChain li mette in `invalid_tool_calls`. Il nodo produce comunque un ToolMessage di errore, così in cronologia non resta una richiesta senza risposta.
- **Eccezione nello strumento**: intercettata e trasformata in testo di errore per il modello, senza crash.

> **Principio di verifica**
> Una risposta plausibile sull'ora **non** dimostra che lo strumento sia stato usato. Servono le evidenze nella traccia: richiesta del modello con nome e argomenti, esecuzione della funzione, risultato, ritorno al modello associato al tool_call_id corretto, risposta che usa quel dato. E va confrontato con una domanda ordinaria, che non deve usare lo strumento.

5. Teoria · Tempo, fusi orari IANA e ISO 8601

- **UTC** è il riferimento universale. Un'ora senza fuso o offset è ambigua: «13:00» dove?
- **Offset**: differenza da UTC, per esempio `+09:00` per Tokyo.
- **Fuso IANA**: nome standard nel formato Area/Città (`Europe/Rome`, `Asia/Tokyo`, `America/New_York`, `UTC`). A differenza di un offset fisso, conosce la storia delle regole e l'**ora legale**: Roma è +01:00 in inverno e +02:00 in estate.
- **ISO 8601**: formato standard di data e ora, es. `2026-10-28T20:08:51+09:00`.
- **zoneinfo** è il modulo della libreria standard Python per i fusi IANA; il pacchetto **tzdata** fornisce il database dei fusi dove il sistema non lo ha (Windows e l'immagine `python:3.13-slim`).

Regola di progetto: l'istante si legge **sempre in UTC** e si converte nel fuso richiesto. Il fuso del computer o del container non influisce. È stato verificato: i log del container erano in UTC (10:25) mentre lo strumento restituiva correttamente le 19:25 di Tokyo.

Anche il **giorno della settimana** viene calcolato nel fuso richiesto (alle 20:00 UTC di giovedì a Tokyo è già venerdì) e preso da una tabella di nomi italiani: `strftime("%A")` dipenderebbe dalle impostazioni di lingua del sistema.

6. Teoria · Docker e Docker Compose

Concetto	Significato
Immagine	Pacchetto immutabile con applicazione, runtime Python e dipendenze, costruito da un Dockerfile a strati (layer).
Container	Istanza in esecuzione di un'immagine, con processo, configurazione e stato propri.
Dockerfile	Ricetta per costruire l'immagine.
compose.yaml	Descrive come eseguire l'applicazione: servizi, build, variabili, rete. Rende l'avvio ripetibile anche con un solo servizio.
.dockerignore	Esclude file dal contesto di build: qui .env (segreti), .venv, cache, .git, tests e docs.
Cache dei layer	Il Dockerfile copia prima requirements.txt e installa, poi copia il codice: se cambia solo il codice, le dipendenze non vengono reinstallate.

```
# Dockerfile
FROM python:3.13-slim
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY assistente/ assistente/
CMD ["python", "-m", "assistente"]
```

```
# compose.yaml
services:
  assistente:
    build: .
    env_file: .env
    environment:
      LMSTUDIO_BASE_URL: http://host.docker.internal:1234/v1
    extra_hosts:
      - "host.docker.internal:host-gateway"
    stdin_open: true    # come -i
    tty: true           # come -t
```

- `PYTHONUNBUFFERED=1`: output e log escono subito, così `docker logs -f` li mostra in tempo reale.
- `environment` prevale su `env_file`: nel container l'indirizzo di LM Studio è sempre quello dell'host.
- `stdin_open` e `tty` servono alla chat interattiva; per una CLI si usa `docker compose run` e non up.

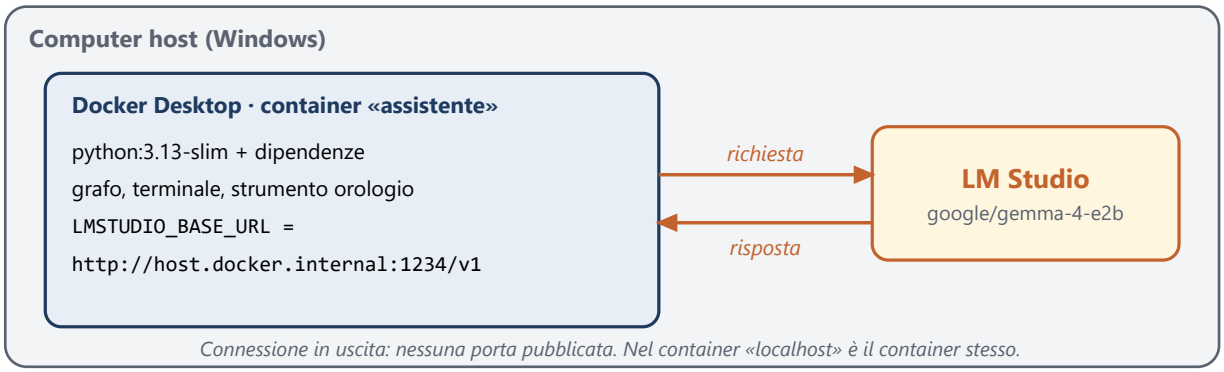


Figura 2 · L'applicazione nel container, LM Studio sull'host. host.docker.internal è il nome con cui Docker Desktop permette al container di raggiungere il computer host.

Comando	A che cosa serve
`docker compose build`	Costruisce l'immagine (da rifare dopo ogni modifica al codice).
`docker compose run --rm --name assistente-demo assistente`	Avvia la chat collegata al terminale; --rm rimuove il container all'uscita.
`docker ps`	Elenca i container attivi (da un secondo terminale).
`docker logs -f assistente-demo`	Segue l'output e i log del container.
`... run --rm -T assistente`	Senza TTY: permette di passare l'input da pipe (usato nella prova automatica).

7. Architettura del progetto

Il codice è diviso in moduli piccoli, ciascuno con una responsabilità (466 righe in tutto). Le lezioni successive estenderanno questo codice senza ripartire da zero: per esempio, un nuovo strumento si aggiunge alla lista `strumenti` passata a modello e grafo.

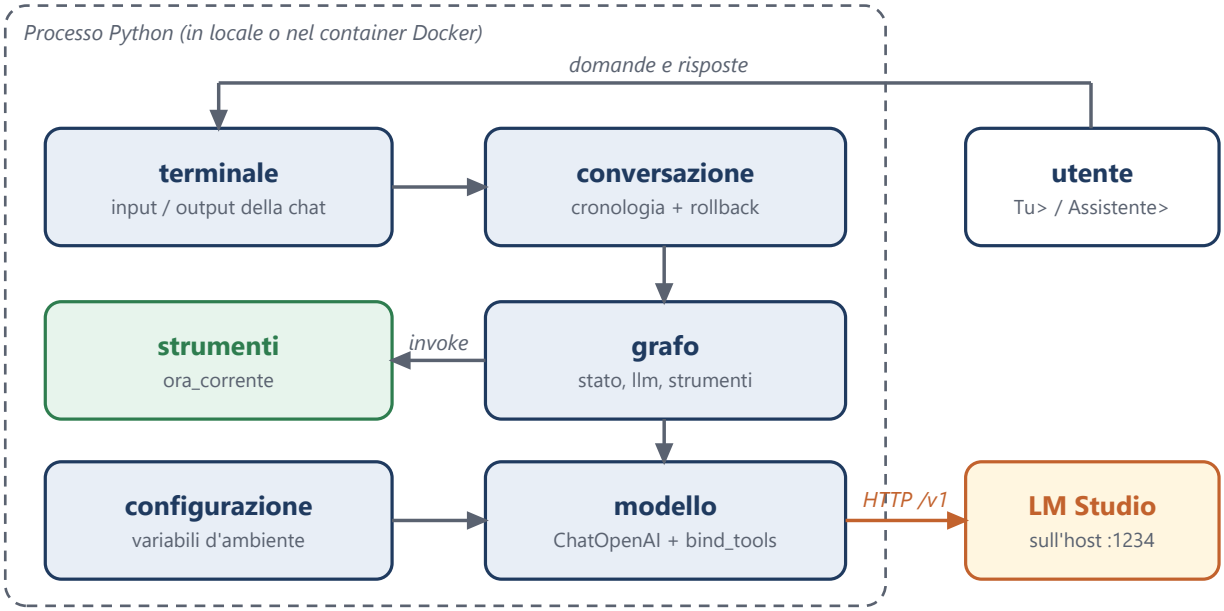


Figura 3 · Moduli e dipendenze.

Modulo	Responsabilità
`configurazione.py`	Legge e valida le variabili d'ambiente; errore chiaro che nomina la variabile.
`strumenti.py`	Strumento `ora_corrente`, validazione dei fusi, formato del risultato.
`modello.py`	ChatOpenAI verso LM Studio con `bind_tools`; controllo di GET /v1/models.
`grafo.py`	Stato, nodi llm e strumenti, `instrada`, limite di passaggi, log.
`conversazione.py`	Cronologia di sessione, rollback del turno, chiamate allo strumento del turno, errori del modello.
`terminale.py`	Ciclo Tu> / Assistente>, righe [strumento], uscita pulita.
`__main__.py`	Avvio: .env, configurazione, log su stderr, verifica del modello, composizione dei pezzi.

Dipendenze (versioni fissate)

Pacchetto	Motivo
langgraph 1.2.18	Grafo esplicito con stato, nodi e instradamento.
langchain-core 1.7.0	Messaggi, strumenti, interfaccia Runnable.
langchain-openai 1.7.3	ChatOpenAI, client compatibile con l'API di LM Studio.
openai 3.39.0	SDK usato da langchain-openai; fissato per riconoscerne gli errori (APIError).
python-dotenv 1.2.4	Carica .env nell'esecuzione locale.
tzdata 2026.5	Database dei fusi IANA, assente nell'immagine slim e su Windows.
pytest 9.1.1 (solo sviluppo)	Test automatici e prove reali con marcatori.

8. Il grafo nel dettaglio

```
class StatoConversazione(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]   # si aggiunge, non si sostituisce
    passaggi: int                                         # chiamate al nodo llm nel turno
    limite_raggiunto: bool

def instrada(stato, max_passaggi) -> Literal["strumenti", "fine"]:
    if not _richieste_di_tool(stato):    # richieste di tool dell'ultimo messaggio
        log.info("Instradamento: risposta finale → fine")
        return "fine"
    if stato.get("passaggi", 0) >= max_passaggi:
        log.info("Instradamento: strumento richiesto ma limite di passaggi raggiunto → fine")
        return "fine"
    log.info("Instradamento: il modello chiede uno strumento → strumenti")
    return "strumenti"

costruttore = StateGraph(StatoConversazione)
costruttore.add_node("llm", llm)
costruttore.add_node("strumenti", nodo_strumenti)
costruttore.add_edge(START, "llm")
costruttore.add_conditional_edges("llm", lambda stato: instrada(stato, max_passaggi),
                                  {"strumenti": "strumenti", "fine": END})
costruttore.add_edge("strumenti", "llm")
grafo = costruttore.compile()
```

Nodo	Che cosa fa
llm	Incrementa `passaggi`; invia al modello [SystemMessage(prompt), *messaggi]; registra a INFO inizio, fine e durata; se il modello chiede ancora uno strumento all'ultimo passaggio consentito imposta `limite_raggiunto` e registra un WARNING.
strumenti	Per ogni richiesta dell'ultimo messaggio cerca lo strumento per nome, controlla gli argomenti, lo esegue e aggiunge un ToolMessage con il tool_call_id della richiesta; registra a INFO richiesta e risultato.

Il limite di passaggi

Con `MAX_PASSAGGI=3` e un modello che chiede sempre lo strumento, il grafo esegue llm 3 volte e strumenti 2 volte, poi si ferma con `limite_raggiunto=True`. La chat mostra «Limite di passaggi raggiunto: nessuna risposta finale. Riformula la domanda.»: l'applicazione non inventa una risposta. Il valore predefinito è 6.

> **Scelta progettuale: niente checkpointer**
> LangGraph può salvare lo stato tra un'invocazione e l'altra con un checkpointer. Qui il grafo è compilato senza: la cronologia vive nell'oggetto Conversazione ed entra nel grafo a ogni turno. Così il rollback di un turno fallito è semplice (si scarta il risultato) e la persistenza, esclusa dai requisiti, non c'è.

9. Un turno passo per passo

Domanda: «Che ore sono adesso a Tokyo?». Messaggi nello stato alla fine del turno:

#	Messaggio	Prodotto da	Contenuto
1	HumanMessage	Conversazione	Che ore sono adesso a Tokyo?
2	AIMessage	nodo llm (passaggio 1)	contenuto vuoto + tool_calls: ora_corrente({"fuso_orario": "Asia/Tokyo"}), id call_…
3	ToolMessage	nodo strumenti	2026-10-28T20:08:51+09:00 (Asia/Tokyo), mercoledì · tool_call_id = id della richiesta
4	AIMessage	nodo llm (passaggio 2)	Adesso sono le 20:08:51 di mercoledì 28 ottobre 2026 a Tokyo.

Log reali con `LIVELLO_LOG=INFO` (prova del 28 ottobre, prima dell'aggiunta del giorno della settimana):

```
11:20:13 INFO assistente.grafo: Inizio chiamata al modello (passaggio 1, 1 messaggi)
11:20:21 INFO assistente.grafo: Fine chiamata al modello in 7.78 s
11:20:21 INFO assistente.grafo: Instradamento: il modello chiede uno strumento → strumenti
11:20:21 INFO assistente.grafo: Richiesta di strumento: ora_corrente({'fuso_orario': 'Asia/Tokyo'})
11:20:21 INFO assistente.grafo: Risultato di ora_corrente: 2026-10-28T19:20:21+09:00 (Asia/Tokyo)
11:20:21 INFO assistente.grafo: Inizio chiamata al modello (passaggio 2, 3 messaggi)
11:20:31 INFO assistente.grafo: Fine chiamata al modello in 9.74 s
11:20:31 INFO assistente.grafo: Instradamento: risposta finale → fine
```

Questa traccia contiene tutte le evidenze richieste dalla slide 13: richiesta del modello (nome e argomenti), esecuzione, risultato, ritorno al modello e risposta che usa il dato. Alla domanda successiva «E a Roma?» il modello ha passato `Europe/Rome`: ha capito il riferimento grazie alla cronologia.

10. Lo strumento orologio

Parte del contratto	Decisione
Nome	`ora_corrente`
Descrizione	Restituisce data, ora e giorno della settimana attuali; da usare ogni volta che servono, senza inventarli; esempi di fusi IANA.
Parametro	`fuso_orario` facoltativo, nome IANA; spazi esterni rimossi. Se assente: `FUSO_ORARIO_PREDEFINITO` (Europe/Rome).
Risultato	`AAAA-MM-GGTHH:MM:SS±HH:MM (Nome/IANA), giorno` · es. `2026-01-15T13:00:00+01:00 (Europe/Rome), giovedì`
Errori	Nessuna eccezione: testo che inizia con `Errore:`, con il valore ricevuto e un esempio IANA (Asia/Tokyo).

```
def ora_corrente_testo(fuso_orario, fuso_predefinito, adesso_utc: datetime) -> str:
    richiesto = fuso_predefinito if fuso_orario is None else fuso_orario
    try:
        zona = valida_fuso(richiesto)      # ZoneInfo, oppure ValueError
    except ValueError:
        return (f"Errore: fuso orario non riconosciuto: {richiesto!r}. "
                f"Usa un nome di fuso IANA, per esempio Asia/Tokyo ({ESEMPI_FUSO}).")
    locale = adesso_utc.astimezone(zona)
    return f"{locale.isoformat(timespec='seconds')} ({zona.key}), {GIORNI[locale.weekday()]}"
```

L'istante è un parametro: in produzione arriva da `datetime.now(timezone.utc)`, nei test è un istante fisso. Così i test confrontano valori stabili (Roma in inverno +01:00 e in estate +02:00, Tokyo, UTC,

cambio di giorno tra UTC e Tokyo). Valori rifiutati senza eccezioni: `Tokyo`, `UTC+2`, stringa vuota, `../etc/passwd`, `Europe`.

11. Cronologia, rollback ed errori

La classe **Conversazione** conserva la cronologia della sessione. A ogni turno invoca il grafo con `[*cronologia, HumanMessage(testo)]`, `passaggi=0` e `limite_raggiunto=False`. La cronologia viene sostituita con i messaggi restituiti **solo se il turno riesce**.

> **Rollback completo del turno**
> Se il modello non risponde o si raggiunge il limite di passaggi, la cronologia torna esattamente com'era prima della domanda: né la domanda, né richieste di tool rimaste senza risposta, né risultati parziali. Così la richiesta successiva a LM Studio è sempre valida e l'utente può riformulare.

Situazione	Comportamento (verificato)
LM Studio non raggiungibile all'avvio	Avviso con l'indirizzo provato; la chat si apre comunque.
LM Studio non raggiungibile durante la chat	Errore: Impossibile contattare il modello su … Controlla che LM Studio sia avviato…; sessione aperta, cronologia invariata.
Errore HTTP del server (es. modello scaricato)	Stesso messaggio chiaro e stesso rollback (openai.APIError copre entrambi i casi).
Modello configurato inesistente	Uscita con codice 1 e l'elenco degli identificativi disponibili.
Configurazione mancante o non valida	Uscita con codice 1 e messaggio che nomina la variabile, senza traceback.
Risposta vuota del modello	Assistente> (il modello non ha restituito testo)
Limite di passaggi	Messaggio esplicito, nessuna risposta inventata, rollback.
Ctrl+C, fine input, esci	Uscita pulita senza traceback, anche durante l'attesa della risposta o all'avvio (codice 130).
Output rediretto su file (Windows)	Nessun crash: i caratteri non codificabili diventano «?».

12. Configurazione

La configurazione arriva solo da variabili d'ambiente. In locale si usa il file `.env`, escluso da Git e creato da `.env.example`, che non contiene credenziali.

Variabile	Obbligatoria	Default	Note
LMSTUDIO_BASE_URL	sì	—	http/https con host, es. http://localhost:1234/v1; nel container è sostituita da host.docker.internal
LMSTUDIO_MODEL	sì	—	Identificativo restituito da GET /v1/models
FUSO_ORARIO_PREDEFINITO	no	Europe/Rome	Validato all'avvio come nome IANA
MAX_PASSAGGI	no	6	Intero ≥ 1
LIVELLO_LOG	no	WARNING	DEBUG, INFO, WARNING o ERROR; log su stderr

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements-dev.txt
Copy-Item .env.example .env
python -m assistente                      # chat locale
$env:LIVELLO_LOG="INFO"; python -m assistente   # con i log delle chiamate
```

13. Test e verifiche

La strategia distingue due livelli, come chiede il CLAUDE.md: **test automatici** ripetibili che non richiedono un modello attivo, e **prove reali** con LM Studio e Docker, tenute separate e attivate esplicitamente con i marcatori pytest. `pytest.ini` esclude le prove reali di default (`-m "not lmstudio and not docker"`).

File di test	N.	Che cosa verifica
test_strumenti.py	18	Orologio con istante fisso: Roma inverno/estate, Tokyo, UTC, default, spazi, fusi non validi, giorno nel fuso giusto, schema inviato al modello.
test_grafo.py	15	Percorsi con modelli finti: solo llm; llm → strumenti → llm; limite (3 llm, 2 strumenti); strumento sconosciuto, argomenti errati o non JSON; log; instrada sui quattro casi; prompt.
test_configurazione.py	14	Default, variabili obbligatorie mancanti, valori non validi (MAX_PASSAGGI, LIVELLO_LOG, fuso, URL).
test_terminale.py	10	esci, righe vuote, fine input e Ctrl+C, errore del modello, risposta vuota, limite, formato della riga [strumento].
test_conversazione.py	6	Due turni senza duplicati; rollback dopo errore di connessione e HTTP; chiamate del turno; rollback al limite.
test_modello.py	3	Verifica del modello: presente, assente con elenco, server irraggiungibile con avviso.
test_avvio.py	3	Configurazione mancante senza traceback; Ctrl+C all'avvio; output non codificabile.
reali/test_lmstudio.py	3	Modello reale: due turni con cronologia («Ada»); ora a Tokyo con chiamata reale e ora nella risposta; domanda ordinaria senza strumento.
reali/test_docker.py	1	Dal container: riga [strumento] con Asia/Tokyo e un'ora valida, e risposta che usa quell'ora.

Modelli finti e istante controllato

- **GenericFakeChatModel** di LangChain e un **RunnableLambda «a copione»** (`tests/finti.py`) restituiscono risposte prestabilite: AIMessage di testo, richieste di tool, oppure eccezioni come un errore di connessione. Così si verificano tutti i percorsi del grafo senza dipendere da LM Studio.
- Lo strumento riceve un **orologio iniettabile**: `orologio=lambda: ISTANTE_INVERNO`.
- Il terminale riceve funzioni `leggi` e `scrivi` iniettabili: i test simulano input ed esaminano le righe stampate.

TDD: test che fallisce, modifica, test superato

Ogni funzionalità è nata da un test scritto prima del codice e visto fallire (RED), poi fatto passare (GREEN). Anche i difetti trovati nelle prove reali o nella revisione sono stati corretti così: prima un test che riproduce il difetto, poi la correzione.

Perché servono anche le prove reali
Una risposta simulata non dimostra che il modello locale sappia usare lo strumento. I test automatici provano che il *programma* gestisce correttamente richieste e risposte; solo la prova con il modello reale prova che *gemma-4-e2b* chiede davvero lo strumento con gli argomenti giusti.

```
python -m pytest                    # 69 passed, 4 deselected
python -m pytest -m lmstudio -v     # 3 passed (richiede .env e LM Studio attivo)
docker compose build
python -m pytest -m docker -v       # 1 passed (richiede Docker e LM Studio)
```

14. Metodo di lavoro: CLAUDE.md, Superpowers e Git

CLAUDE.md

File di istruzioni stabili per Claude Code nella cartella del progetto: obiettivo e confini (solo questa cartella, niente MCP/RAG/web anticipati), scelte tecniche (LangGraph, LM Studio sull'host, Docker Compose, configurazione da ambiente, dipendenze fissate), metodo (Superpowers, niente codice prima di specifica e piano approvati), test, Git (Conventional Commits in italiano, staging selettivo, push solo su richiesta) e documentazione (README con soli comandi verificati).

Il ciclo di Superpowers applicato al progetto

Fase	Che cosa è stato prodotto
Brainstorming	Decisioni chiarite con lo studente: fuso selezionabile, chiamate visibili in chat e nei log, log silenziosi di default, cronologia persa alla chiusura.
Specifica	docs/superpowers/specs: comportamento, contratto dello strumento, errori, test, corrispondenza con la consegna; requisiti con ID (RF, RE, RV).
Piano	Cinque incrementi con file, interfacce, test da scrivere prima e verifiche attese: grafo fisso, LM Studio e cronologia, orologio e instradamento, Docker, verifica completa.
Implementazione	Esecuzione del piano in linea con TDD, un commit per incremento verificato, registro delle decisioni.
Review e verifica	Revisione indipendente di tutto il lavoro: nessun problema critico; corretti 5 problemi importanti, ognuno partendo da un test che falliva.
Integrazione	README e CLAUDE.md aggiornati, handoff per riprendere il lavoro, push lasciato allo studente.

Commit (Conventional Commits)

Commit	Messaggio
fc76c65	docs(spec): aggiungi specifica e requisiti e sposta CLAUDE.md nel progetto
ac06cbd	feat(grafo): aggiungi la struttura del progetto e il grafo con risposta fissa
ece47a7	docs(piano): aggiungi il piano di implementazione approvato
d8f214c	feat(modello): collega LM Studio e conserva la cronologia della sessione
da96444	feat(terminale): aggiungi la chat da terminale e l'avvio
fe86f84	feat(strumenti): aggiungi lo strumento orologio con fuso orario esplicito
4342b15	docs: aggiungi l'handoff della sessione

Commit	Messaggio
500c58f	feat(grafo): instrada le richieste di tool verso lo strumento orologio con limite di passaggi
3f1db2c	feat(docker): avvia l'assistente con Docker Compose e LM Studio sull'host
5dc02b7	fix(grafo): restituisci un errore al modello per le richieste di tool con argomenti non validi
63d08d9	fix(avvio): evita traceback con URL non valido, Ctrl+C all'avvio e output rediretto
865945d	test(docker): verifica che la risposta dal container usi l'ora restituita dallo strumento
fe0085e	feat(strumenti): aggiungi il giorno della settimana al risultato dell'orologio
bbe18d9	docs: completa README e CLAUDE.md dopo la verifica finale

15. Risultati, misure e decisioni

Affidabilità del tool calling con il modello reale

Versione	Ora a Tokyo con fuso_orario=Asia/Tokyo	Domanda ordinaria senza strumento
Descrizione dello strumento senza descrizione del parametro	12 su 17 (in 5 casi chiamata senza fuso: ora di Roma)	9 su 9
Con descrizione del parametro e frase nel prompt di sistema	12 su 12	12 su 12

Il modello ha sempre chiamato lo strumento (mai un'ora inventata); il problema era l'argomento mancante. Il test reale non è stato ammorbidito: si è migliorato il contratto. Il modello resta non deterministico, quindi un errore occasionale va segnalato, non nascosto.

Giorno della settimana

In una prova il modello ha scritto «lunedì 28 ottobre» (era mercoledì): il giorno non era nel risultato e il modello lo calcolava da sé. Su decisione dello studente il giorno è stato aggiunto al risultato e la specifica aggiornata. Nelle prove successive il modello ha usato il giorno corretto restituito dallo strumento.

Dove va il tempo di risposta

Richiesta	Totale	Primo token	Generazione	Token generati
Ora a Tokyo, 1a chiamata (richiesta dello strumento)	6,8 s	0,6 s	4,4 s	63
Ora a Tokyo, 2a chiamata (risposta)	8,6–9,2 s	0,8 s	6,3–6,9 s	90–99
Capitale della Francia	12,5–14 s	0,3–0,8 s	10–11,6 s	157–179
Ora a Tokyo dopo 10 turni di cronologia	13,8 s	7,3 s	11,3 s	62

- **Causa principale: ragionamento nascosto**. Per la capitale della Francia, 176 token su 188 sono *reasoning_tokens*; la risposta visibile ne usa circa 8.
- **Generazione a circa 16 token/s** e lettura del prompt lenta con cronologie lunghe: probabile esecuzione in buona parte su CPU (da verificare il GPU Offload in LM Studio).
- Lo strumento impiega millisecondi; Docker non aggiunge tempi apprezzabili.
- Con il parametro `reasoning_effort="none"` la stessa domanda è scesa da 11,9 s a 3,6 s (0 token di ragionamento). Non è stato adottato: l'affidabilità 12 su 12 è stata misurata con il ragionamento

attivo e andrebbe rimisurata prima di cambiare.

Revisione indipendente

Esito: «pronto con correzioni», nessun problema critico, i cinque punti del focus di revisione soddisfatti. Corretti: richieste con argomenti non JSON ignorate; prova Docker che passava anche con un errore dello strumento; traceback con URL malformato, con Ctrl+C all'avvio e con output rediretto.

16. Limiti, problemi aperti e prossimi passi

Punto aperto	Stato
Chat Docker con TTY vero (comando della slide 17 digitando a mano)	Provata solo con input da pipe; da confermare in un terminale interattivo.
LM Studio spento davvero	Simulato con una porta chiusa; comportamento coperto anche dai test.
Ctrl+C in una chat interattiva	Coperto dai test automatici, non provato dal vivo.
La prova Docker non ricostruisce l'immagine	Eseguire prima docker compose build (scritto in CLAUDE.md).
Risposta del modello fatta di soli spazi	Stampa Assistente> vuoto invece del testo previsto (minore).
Tempi di risposta	Dominati dal ragionamento nascosto del modello; possibile ottimizzazione da decidere.
Push	Da eseguire dallo studente.

Prossime lezioni

Il progetto è la base delle lezioni successive: altri strumenti, MCP, workflow e RAG estenderanno questo codice e questo grafo. I punti di estensione sono già presenti: la lista `strumenti` (un nuovo strumento è un altro BaseTool), il nodo strumenti generico che cerca per nome, lo stato del grafo a cui aggiungere campi, la configurazione centralizzata.

17. Come impostare l'esposizione

Scaletta suggerita (circa 12 minuti)

Tempo	Parte	Messaggio chiave
1 min	Obiettivo	Chatbot locale con grafo esplicito, strumento orologio reale, Docker. Requisiti della slide 26.
2 min	Teoria LangGraph	Stato, nodi, archi, reducer add_messages; dal grafo con risposta fissa al ciclo con strumenti (Figura 1).
2 min	Teoria tool calling	Il modello chiede, il programma esegue; la descrizione è il contratto; tool_call_id.
1 min	Architettura	Moduli piccoli e separati (Figura 3); LM Studio sull'host (Figura 2).
3 min	Demo	Domanda ordinaria, domanda sull'ora con riga [strumento], log INFO, avvio in Docker.
2 min	Test e metodo	69 test senza modello, prove reali separate, TDD, Superpowers, Conventional Commits.
1 min	Risultati e limiti	12/17 → 12/12, giorno della settimana, tempi e ragionamento nascosto, punti aperti.

Script della demo

```
# 1. Locale, con i log delle chiamate
.\.venv\Scripts\Activate.ps1
$env:LIVELLO_LOG="INFO"
python -m assistente
Tu> Qual è la capitale della Francia?     (nessuna riga [strumento])
Tu> Che ore sono adesso a Tokyo?          (riga [strumento] + log di instradamento)
Tu> E a Roma?                             (la cronologia fa capire il riferimento)
Tu> esci

# 2. Docker (LM Studio resta sull'host)
docker compose build
docker compose run --rm --name assistente-demo -e LIVELLO_LOG=INFO assistente
#    da un secondo terminale:
docker ps
docker logs -f assistente-demo

# 3. Test
python -m pytest
```

> **Consigli pratici**
> Avviare LM Studio e caricare il modello prima della demo (la prima chiamata è più lenta). Avvisare che ogni risposta richiede alcuni secondi e spiegare perché (ragionamento nascosto, due chiamate per le domande sull'ora). Tenere pronta una traccia di log già ottenuta, nel caso il modello rallenti.

18. Domande probabili e risposte

Perché usare LangGraph e non un semplice ciclo while?
Perché il flusso diventa esplicito e verificabile: stato, nodi e instradamento sono riconoscibili, i test possono controllare quali nodi sono stati eseguiti, e aggiungere strumenti o passaggi nelle prossime lezioni non richiede di riscrivere il ciclo.

Il modello esegue la funzione dell'orologio?
No. Il modello produce solo una richiesta strutturata (nome e argomenti JSON). La funzione Python viene eseguita dal nodo strumenti e il risultato torna al modello come ToolMessage.

Come fate a sapere che l'ora non è inventata?
Dalla traccia: riga [strumento] in chat e log INFO con richiesta, esecuzione e risultato; il test reale controlla che la risposta contenga l'ora HH:MM restituita dallo strumento.

Dove viene conservata la memoria della conversazione?
Nell'applicazione, non nel modello: la classe Conversazione tiene la lista dei messaggi e la reinvia a ogni turno. Il reducer add_messages aggiunge le nuove risposte senza sostituire la cronologia. Alla chiusura viene persa.

Che cosa succede se LM Studio è spento?
All'avvio compare un avviso; durante la chat un errore con l'indirizzo provato e un suggerimento. La sessione resta aperta e la cronologia torna com'era prima della domanda. Nessuna risposta inventata.

A che cosa serve il limite di passaggi?
A interrompere cicli che non terminano, per esempio un modello che chiede lo strumento all'infinito. Al limite il turno finisce con un messaggio esplicito e il rollback.

Perché il fuso orario è un parametro e non fisso?
Per rispondere su qualsiasi luogo («che ore sono a Tokyo?»). Il parametro è validato come nome IANA e un valore non valido produce un errore testuale per il modello, non un'eccezione.

Perché l'ora non dipende dal fuso del container?

Perché l'istante si legge in UTC e si converte nel fuso richiesto con zoneinfo e tzdata. Nel container (in UTC) lo strumento restituiva correttamente l'ora di Tokyo.

Come comunica il container con LM Studio?

Con una connessione in uscita verso http://host.docker.internal:1234/v1, il nome con cui Docker Desktop raggiunge l'host. Non serve pubblicare porte; localhost nel container indicherebbe il container stesso.

Come si testa un grafo senza modello?

Con modelli finti che restituiscono risposte prestabilite (testo, richieste di tool, eccezioni) e con un istante fisso per l'orologio. I test verificano percorsi, ToolMessage, limite, rollback e log.

Perché le risposte sono lente?

Il modello genera prima un ragionamento nascosto di circa 140–195 token a circa 16 token/s; le domande sull'ora richiedono due chiamate. Con reasoning_effort="none" una risposta è scesa da 11,9 a 3,6 s, ma va rimisurata l'affidabilità del tool calling prima di adottarlo.

Che cosa avete cambiato dopo le prove reali?

La descrizione del parametro fuso_orario e il prompt (12/17 → 12/12) e il giorno della settimana nel risultato. I test non sono stati ammorbiditi: è stato migliorato il contratto.

Come si aggiunge un nuovo strumento?

Si crea un altro BaseTool e lo si aggiunge alla lista strumenti passata a crea_modello e crea_grafo: il nodo strumenti lo trova per nome senza altre modifiche al grafo.

19. Glossario

Termine	Definizione
Agente	Programma in cui un LLM può decidere di usare strumenti in un ciclo, fino a una risposta finale.
Arco condizionale	Collegamento del grafo il cui nodo di destinazione è scelto da una funzione sullo stato.
Checkpointer	Componente LangGraph che salva lo stato tra invocazioni (non usato nel progetto).
Contesto	Numero massimo di token che il modello considera in una chiamata (qui 8192).
Conventional Commits	Convenzione dei messaggi di commit: tipo(ambito): descrizione, es. feat(grafo): …
GGUF	Formato di file per modelli quantizzati eseguiti da llama.cpp e LM Studio.
IANA (fuso)	Database standard dei fusi orari, con nomi come Europe/Rome e regole dell'ora legale.
ISO 8601	Standard di rappresentazione di data e ora, es. 2026-10-28T20:08:51+09:00.
JSON Schema	Formato con cui si descrivono i parametri di uno strumento al modello.
Quantizzazione	Riduzione della precisione dei pesi (es. a 4 bit) per ridurre memoria e calcolo.
Reducer	Funzione che combina l'aggiornamento di un nodo con lo stato esistente (add_messages).
Rollback	Ripristino della cronologia com'era prima di un turno fallito.
Runnable	Interfaccia LangChain con invoke/stream, comune a modelli, strumenti e grafi compilati.
TDD	Test-Driven Development: test che fallisce, codice minimo che lo fa passare, poi pulizia.
Token	Frammento di testo su cui il modello lavora; si misura la velocità in token al secondo.
Tool calling	Meccanismo con cui il modello richiede in forma strutturata l'uso di una funzione del programma.

Termine	Definizione
tool_call_id	Identificativo che associa il risultato di uno strumento alla richiesta del modello.
TTY	Terminale interattivo; in Docker si attiva con -it o stdin_open + tty.